GIT UNDO, RECOVERY, AND DEBUGGING

By Hayder Ben Abdelhamid | June 2026

```
C:\Users\MSI\git-learning-log>git log --oneline -5
29b547d (HEAD -> main, origin/main, origin/HEAD) Revert "Add incorrect workflow note"
60023e8 Add incorrect workflow note
263ee19 Add cherry-pick section
6b2dae4 Add reflog section
76651b3 Add revert section
```

Project Overview: Mastering Git Recovery and Debugging

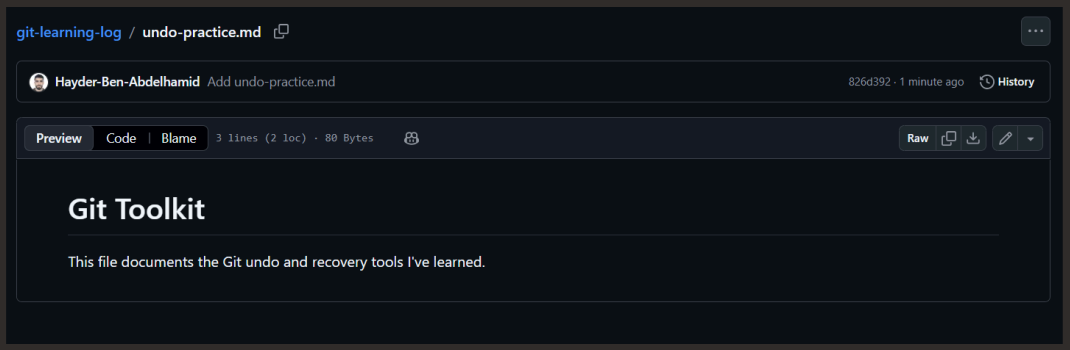
What this project set out to achieve

In this project, I'm learning how to look throught my git history to revert and reset changes, identify bugs and more.

Setting Up the Repository and Environment

Goals for this setup step

In this step, I'm setting up My tools and repository so that I can get started with undoing, recovering and debugging with Git.



Configuring Git for this project

I configured git because I need my tools (git,GitHub, VScode) to learn the Git concepts in this projects.

Building a Meaningful Commit History

Why a rich history matters for practice

In this step, I'm building multiple documentation sections in my hit-learning-log folder so that I can practice different git concepts, and then i will push all the commits to GitHub and create a magnificent commit history.

```
C:\Users\MSI\git-learning-log>git log --oneline
263ee19 (HEAD -> main, origin/main, origin/HEAD) Add cherry-pick section
6b2dae4 Add reflog section
76651b3 Add revert section
096a35b Add reset section
826d392 Add undo-practice.md
```

The value of granular commits

I committed separately because I wanted to:
1. Build a commit history, so I wanted multiple commits.
2. I wanted to add a section one by one so that I know what best practice looks like when you add chunks of code, you want to be able to document it and have comit messages.

Undoing Mistakes Safely with Reset and Revert

Approach to undoing bad commits

In this step, I'm learning how to undo a bad commit so that I can go back and corruct any mistakes. I can use either reset or revert to do this depending on whether the changes have been made in my local dicretory or the public directory.

Reset vs. revert: choosing the right tool

I would use git reset when I want to undo a commit, but i haven't pushed it to GitHub yet and git revert when I want to undo a commit, but I have pushed into GitHub.

Recovering Lost Commits with Git Reflog

Simulating and recovering from data loss

In this step, I'm simulating lost work so that I can prove that git reflog is useful in finding a lost commit.

```
29b547d (HEAD -> main, origin/main, origin/HEAD) HEAD@{0}: reset: moving to HEAD~1
a4bf584 HEAD@{1}: commit: Add bisect section with tips
29b547d (HEAD -> main, origin/main, origin/HEAD) HEAD@{2}: revert: Revert "Add incorrect workfl
ow note"
60023e8 HEAD@{3}: commit: Add incorrect workflow note
263ee19 HEAD@{4}: reset: moving to HEAD~1
7b44b76 HEAD@{5}: commit: Add dangerous advice
263ee19 HEAD@{6}: commit: Add cherry-pick section
6b2dae4 HEAD@{7}: commit: Add reflog section
76651b3 HEAD@{8}: commit: Add revert section
096a35b HEAD@{9}: commit: Add reset section
826d392 HEAD@{10}: commit: Add undo-practice.md
1aeb2f2 HEAD@{11}: checkout: moving from stash-practice to main
6795574 (origin/stash-practice, stash-practice) HEAD@{12}: commit: Add stash lesson to conflict
 practice
```

How reflog sees what git log cannot

Git reflog can find the lost commit because it stores every psotion that the head that Git has been on.

```
## Bisect

- git bisect start: begin a binary search session
- git bisect bad: mark current commit as containing the bug
- git bisect good <ref>: mark a known-good commit
- Git checks out middle commits; you test and mark good/bad
- git bisect reset: end the session and return to original HEAD
```

Cherry-Picking a Hotfix Across Branches

Setting up the cherry-pick scenario

In this step, I'm setting up a feature branch with a mix of feature work and a bug fix so that i can simulate a production hot fix worldflow from start to finish.

```
C:\Users\MSI\git-learning-log>git log --oneline feature/new-docs
0cd95a7 (feature/new-docs) Add advanced tips
d093ed6 Fix typo in reset description
735ad22 Add collaboration section
a4bf584 Add bisect section with tips
29b547d Revert "Add incorrect workflow note"
60023e8 Add incorrect workflow note
263ee19 Add cherry-pick section
6b2dae4 Add reflog section
76651b3 Add revert section
096a35b Add reset section
826d392 Add undo-practice.md
1aeb2f2 Quick fix on main
5415f90 Add git stash lesson and interactive rebase preference (#5)
f7ae9ee (feature/rebase-practice) Add lessons learned section
```

When cherry-pick beats a full merge

I would use cherry-pick instead of merge because i only want one or a select commit to be merged into my main branch.

Hunting Down a Bug with Git Bisect

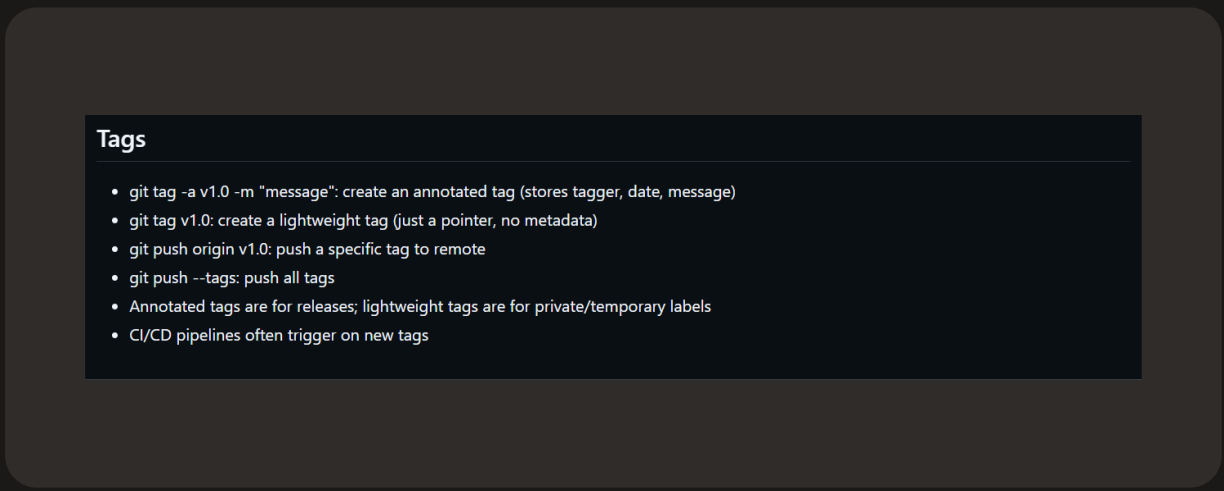
Binary search through commit history

In this step, I'm using Get bisect to do a binary search through all the commit history that i have to identify a bad commit and then remove that.

Identifying the first bad commit

Git bisect identified the commit with message which was 6fe404d2d352349f11cd40ac67aabee66cf01505 is the first had commit. This is usually hard to find manuaaly because it might be hidden somewhere in hundreds of commits, and git biscet uses binary search to go throught and identify where the error or the bad commit was.

Tagging a Release for CI/CD Readiness



Annotated vs. lightweight tags

In this project extension, I learned that annotated tags store tagger name, date and a message (meant for releases). Lightweight tags are just pointers with no metadata (meant for private or temporary labes).